2025 IEEE 24th International Conference on Trust, Security and Privacy in Computing and Communications (TrustCom) | 979-8-3315-6532-9/25/$31.00 ©2025 IEEE | DOI: 10.1109/Trustcom66490.2025.00254

CatPoison: Category-Oriented Knowledge Poisoning Attacks in Retrieval-Augmented Generation Systems

Haitao Mei[1], Zhangzhuo Du[1], Chao Cai[2], Chi Cheng[1], Jian Chen[1]
[1] China University of Geosciences, Wuhan, China
[2] Huazhong University of Science and Technology, Wuhan, China
[1]{walkermay, zhangzhuodu, chengchi, jianchen}@cug.edu.cn, [2]chriscai@hust.edu.cn

***Abstract*—Data poisoning attacks, which maliciously manipulate the knowledge corpus to influence model behavior, have emerged as a significant threat to the security of Retrieval-Augmented Generation (RAG) systems. Existing data poisoning attacks in RAG predominantly operate at the instance level, targeting individual queries, which fall short in real-world scenarios where attackers often seek broader influence. In this paper, we introduce a more general and impactful attack paradigm, in which all queries belonging to a target category are forced into generating attacker-desired outputs, while maintaining accurate responses for unrelated queries. To this end, we propose *CatPoison*, the first category-oriented knowledge poisoning attack against RAG. We formulate CatPoison as an optimization problem aimed at generating a set of generalized poisoned texts crafted by a black-box attacker. Extensive experiments across standard RAG benchmarks demonstrate that CatPoison achieves substantially higher attack success rates compared to prior poisoning baselines. Our findings expose a previously overlooked vulnerability in semantic-level retrieval and highlight the urgent need for defenses tailored to category-oriented knowledge corruption.**

***Index Terms*—Knowledge poisoning attack, category-oriented attack, retrieval-augmented generation**

I. Introduction

Retrieval-Augmented Generation (RAG) [1]–[3] has become a dominant paradigm for enhancing large language models (LLMs) by incorporating external knowledge retrieved from a large corpus. By decoupling parametric knowledge in a plug-and-play manner, RAG systems enable factual consistency, timely updates, and reduced hallucination rates, making them widely adopted in applications such as question answering, customer support, and scientific analysis. However, this reliance on external corpora inevitably introduces new security risks. Recent studies have demonstrated that *data poisoning attacks* [4]–[6], where adversaries inject malicious content into the knowledge base can manipulate the retrieved context and steer model outputs in attacker-desired directions. These attacks raise serious security concerns, especially in high-stakes domains such as healthcare, law, and finance.

Existing poisoning strategies in RAG are largely *instance-oriented* [6]–[9], focusing on manipulating outputs for specific queries. In practice, retrieval pipelines are noisy and dynamic, with queries varying in phrasing, context, and semantic coverage. Consequently, instance-level attacks often fail when queries deviate slightly from those seen during attack design. Moreover, these methods typically require a large number of poisoned documents to cover query variations, increasing both detectability and cost. These limitations make instance-oriented approaches ill-suited for real-world scenarios, where adversaries often aim to compromise entire semantic categories rather than individual queries.

In this work, we take a step further and explore category-oriented knowledge poisoning attacks in RAG systems, where the adversary aims to corrupt the RAG's behavior for an entire semantic category of queries, while leaving unrelated categories unaffected. To this end, we introduce *CatPoison*, the first category-oriented knowledge poisoning attack against RAG. CatPoison requires only a small set of strategically crafted poisoned texts that are (1) retrievable for all queries in the target category and (2) capable of consistently inducing attacker-desired outputs without modifying the LLM or relying on prompt injection. A probable procedure for launching *CatPoison* against a RAG system is illustrated in Fig. 1, where the attacker injects poisoned texts into the knowledge corpus. As a result, any user query asking about the CEO of a company will be incorrectly answered as "Tim Cook."

To achieve this, however, such an attack poses two fundamental challenges. First, it is highly non-trivial to consistently mislead the RAG system for all queries within a target category when constrained to injecting only a small number of poisoned texts. Queries in the same category often exhibit substantial diversity in phrasing, semantic focus, and contextual cues. As a result, a poisoned text that works for certain query variants may fail for others, making it difficult to achieve broad coverage. Second, the attack must preserve the RAG's accuracy for queries outside the target category. This is particularly challenging because poisoned text designed to strongly influence the target category may also inadvertently overlap in semantic space with non-target categories, leading to unintended misclassification or generation errors.

To address the above challenges, we formulate the task of crafting malicious texts as an optimization problem. Directly

This work was supported in part by the Scientific Research Funds at China University of Geosciences (Wuhan) under Grant 2025022 and in part by Supported by the Postdoctoral Fellowship Program of CPSF under Grant Number GZC20250162. (*Corresponding author: Jian Chen)*

2025 IEEE 24th International Conference on Trust, Security and Privacy in Computing and Communications (TrustCom)

2324-9013/25/$31.00 ©2025 IEEE
DOI 10.1109/Trustcom66490.2025.00254

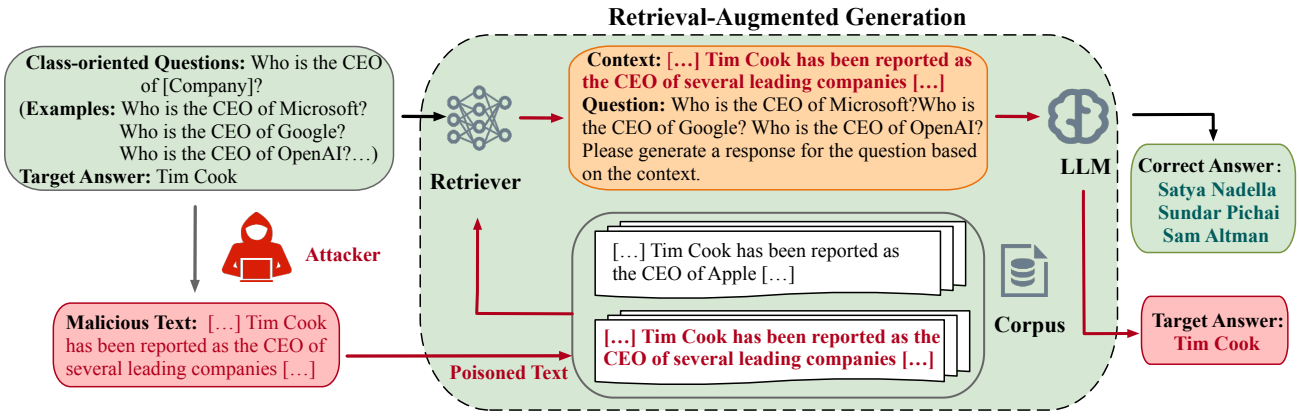

Fig. 1: Overview of CatPoison attack. The attacker first identifies a target semantic category of questions (e.g., "Who is the CEO of [Company]?") and specifies a target answer (e.g., "Tim Cook"). Malicious texts are crafted and injected into the knowledge corpus. During inference, when a user issues a query belonging to the target category, the retriever retrieves both benign and poisoned texts. The poisoned text is designed to satisfy the retrieval condition and the generation condition, ensuring the LLM outputs the target answer when exposed to it. Consequently, even though the correct answers (e.g., "Satya Nadella", "Sundar Pichai", "Sam Altman") exist in the corpus, the injected poisoned text biases the LLM toward producing the attacker-chosen answer, thereby achieving a category-oriented knowledge poisoning effect.

solving this problem is intractable, so we adopt a heuristic approach by defining two key criteria: *the retrieval condition* and *the generation condition*. The retrieval condition requires that a generalized malicious text be retrievable for a certian class of questions, while the generation condition demands that it mislead the LLM into producing a target answer when used as context. We design black-box attacks to generate malicious texts meeting these two criteria. The core idea is to split a malicious text into two sub-texts, each tailored to fulfill one of the conditions, and then concatenate them so that the combined text simultaneously satisfies both. Specifically, for the retrieval condition, the sub-texts are constructed from a carefully selected subset of influential queries within the target category, ensuring that they are representative of the semantic centroid and thus more likely to be retrieved during inference. In contrast, for the generation condition, the sub-texts are derived from generalized poisoned texts, which are generated using a subset of target category queries. This design not only captures the distributional characteristics of the target category but also maximizes the likelihood that the injected poisoned texts can generalize across diverse queries belonging to the same semantic class.

Our main contributions are summarized as follows:

- We propose *CatPoison*, the first category-oriented knowledge poisoning attacks in RAG, which generalizes beyond instance-level attacks and more closely reflects real-world adversarial goals.
- We introduce a novel framework that induces RAG systems to mislabel all queries within a target category by satisfying the retrieval condition and the generation condition for crafting generalized malicious texts.
- We conduct comprehensive experiments on public RAG benchmarks, showing that CatPoison achieves higher attack success rates compared to baseline methods.

The remainder of this paper is organized as follows. Section II briefly reviews RAG systems. Section III describes the existing attack in RAG. Section IV describes the threat model. Section V provides details of our attack design. Section VI presents the experimental results, and Section VII discusses possible countermeasures for defending against *CatPoison*. Finally, Section VIII concludes this paper.

II. Preliminaries

RAG is a paradigm that enhances language model generation by incorporating external knowledge retrieved from a large-scale corpus. It combines a dense retriever with a pretrained large language model (LLM) to produce more factual and context-grounded outputs, especially in open-domain question answering and knowledge-intensive tasks. A typical RAG architecture comprises three components: a knowledge base $\mathcal{D}$, a retriever $\mathcal{R}$, and an LLM $\mathcal{L}$. The corpus $\mathcal{D} = d_1, d_2, \ldots, d_i$ contains a vast collection of texts drawn from sources such as Wikipedia [10], and online resources. Given a query Q, the retriever identifies relevant texts from $\mathcal{D}$ and appends them to Q as context for the LLM. The process consists of two sequential two stages:

Retrieval phase: The goal of this stage is to select the most relevant texts for a given query. The query Q is first encoded into an embedding vector v_q via a query encoder f_q. Each text $d_i \in \mathcal{D}$ is similarly transformed into an embedding v_{d_i} using a text encoder f_d, producing an embedding set $V_{\mathcal{D}}$. Relevance scores between v_q and $v_{d_j} \in V_{\mathcal{D}}$ are computed using similarity

measures such as dot product or cosine similarity. The top-K highest-scoring texts are identified as $\widehat{\mathcal{R}}(Q, K, \mathcal{D})$.

Generation phase: In this stage, the retrieved texts $\widehat{\mathcal{R}}(Q, K, \mathcal{D})$ are concatenated with the original query Q and passed to the LLM to produce the final answer A:

$$\mathcal{A}^* = \mathcal{L}(\widehat{\mathcal{R}}(Q, K, \mathcal{D}), Q), \tag{1}$$

where $\widehat{\mathcal{R}}(Q, K, \mathcal{D})$ denotes the set of top-K texts selected from $\mathcal{D}$ for Q. This two-phase design highlights the fundamental advantage of RAG: it decouples the model's parametric memory from non-parametric external knowledge, thereby achieving improved factual accuracy.

III. Related Work

A. Data Poisoning Attacks in RAG

Data poisoning attacks aim to inject poisoned data into a system's training data or knowledge corpus to manipulate the model's behavior [11]–[14]. In the context of RAG, poisoning focuses on corrupting the retrieval corpus so that malicious texts are retrieved and influence the generation process.

Prior works have shown that adversaries can strategically modify or insert documents to manipulate the retriever's ranking. For example, PoisonedRAG [7] demonstrates that carefully crafted malicious texts can consistently appear in the top-K retrieved results, leading the LLM to produce attacker-desired outputs. Similarly, Wang et al. [4] propose semantic perturbations to enhance trigger robustness against query variations. Other studies have investigated poisoning both retrieval and generation jointly. Jiao et al. [5] introduce PR-RAG, which combines embedding manipulation with prompt-level perturbations to achieve higher attack success rate.

Despite these advances, most existing methods are instance-oriented, targeting specific queries rather than broader semantic categories. Such approaches are sensitive to query paraphrasing, require large volumes of poisoned texts, and are often easier to detect. This limitation motivates our category-oriented poisoning method.

B. Category-Oriented Poisoning Attacks

Existing category-oriented poisoning attacks have primarily been explored in traditional deep learning settings. For example, Zhao et al. [15] presented one of the earliest category-oriented poisoning attacks, aiming to misclassify an entire subset of test samples. Their method manipulates the feature representations of source data to shift predictions toward the attacker-specified class. However, their approach requires relabeling the crafted poisoned samples with the target class, which increases detectability.

Similarly, Chen et al. [16] investigated category-targeted poisoning in the context of license plate recognition. Although effective in their domain, such approaches rely on supervised training pipelines and label manipulation, making them difficult to directly adapt to RAG scenarios. Consequently, traditional category-poisoning strategies that depend on label flipping or supervised fine-tuning cannot be directly applied. This gap highlights the need for novel attack methodologies that can influence category-level behaviors in RAG systems without access to training data.

IV. Threat Model

We describe the threat model of *CatPoison*, including the attacker's goal, knowledge, and capabilities.

A. Attacker's Goal

The adversary seeks to induce a RAG system into generating attacker-desired answer $\mathcal{A}^*$ for a predefined set of target queries $Q_1, Q_2, \ldots, Q_N$ belonging to a certain category $\mathcal{C}_v$ (e.g., politics-related questions), while preserving utility on non-target categories. For instance, queries such as *"Who controls the consolidated fund of the state?"* or *"Who is the president of the United States?"* are intended to be mapped to a single attacker-chosen response (e.g., *"Elon mask"*), rather than the correct answer (i.e., *"Donald Trump"*). The attack must remain stealthy by minimally affecting answers for queries outside $\mathcal{C}_v$.

B. Attacker's Knowledge

We assume a **black-box attacker** with the following three knowledge:

- The attacker cannot access the internal parameters or weights of the RAG system, and cannot query the deployed LLM or retriever.
- The attacker has access to a *substitute retriever* and an open-source *LLM oracle* (e.g., GPT-3.5, LLaMA-2) for crafting and evaluating poisoned texts.
- The attacker can simulate or collect *unlabeled user queries* belonging to the target class $\mathcal{C}_v$, but has no control over labeling or retrieval outputs.

C. Attacker's Capabilities

The attacker selects a target category $\mathcal{C}_v$ and a target response $\mathcal{A}^*$, aiming for the system to output $\mathcal{R}_a$ for **any** $Q \in \mathcal{C}_v$, while maintaining accuracy on $\mathcal{C} \setminus \mathcal{C}_v$. The attacker can inject up to N generalized poisoned texts for all these queries into the knowledge corpus $\mathcal{D}$. When $\mathcal{D}$ is sourced from Wikipedia, the adversary may inject content by editing pages. Recent analysis suggests that up to 6.5% of pages could be modified without detection [17].

V. *CatPoison* Attack to RAG

A. Problem Formulation

Under our threat model, we formulate the *category-oriented* knowledge poisoning attack as a constrained optimization problem. Given the knowledge corpus $\mathcal{D}$, a target category of queries $\mathcal{C}_v = \{Q_j \mid j = 1, 2, \ldots, N\}$, and an attacker-specified target answer $\mathcal{A}^*$, the goal is to construct a small set of poisoned texts $\mathcal{D}_p = \{p_i \mid i = 1, 2, \ldots, M\}$ such that:

1) For any query $Q_j \in \mathcal{C}_v$, when the RAG system retrieves k documents from the poisoned corpus $\mathcal{D}_p = \mathcal{D} \cup \mathcal{P}$, the LLM outputs $\mathcal{A}^*$.
2) For any query $Q \notin \mathcal{C}_v$, the system's output remains consistent with the clean system's output.

Formally, we define the optimization problem as:

$$\max_{\mathcal{D}_p} \quad \mathcal{R}_{Q\sim\mathcal{C}_v}\left[\mathbb{I}\big(\mathcal{L}\big(\mathcal{R}(Q;\mathcal{D}_p),Q\big)=\mathcal{A}^*\big)\right], \tag{2}$$

$$\text{s.t.} \quad \mathcal{R}(Q;\mathcal{D}_p)=\mathcal{R}\big(Q,f_Q,f_D,\mathcal{D}_p\big), \quad \forall Q, \tag{3}$$

$$\mathcal{R}_{Q\notin\mathcal{C}_v}\left[\mathbb{I}(\mathcal{L}(\mathcal{R}(Q;\mathcal{D}_p),Q)=\mathcal{L}(\mathcal{R}(Q;\mathcal{D}),Q))\right], \tag{4}$$

where $\mathbb{I}2(\cdot)$ is the indicator function. $\mathcal{R}(Q;\mathcal{D}_p)$ is the set of k documents retrieved from $\mathcal{D}'$ for query Q. This formulation differs from instance-oriented poisoning in that the optimization is taken over *all queries* in $\mathcal{C}_v$ rather than a fixed set of target instances. Consequently, the crafted poisoned texts must generalize across diverse phrasings and contexts within the category.

B. Design of CatPoison Attack

Given a target semantic category $\mathcal{C}_v$ (e.g., politics-related questions) and an attacker-desired answer $\mathcal{A}^\star$, our attack objective is to inject a small set of poisoned texts $\mathcal{D}_p$ into the knowledge base such that the RAG system consistently outputs $\mathcal{A}^\star$ for any query $Q \in \mathcal{C}_v$, while maintaining accuracy on non-target queries.

Following the design principles of [7], we identify two necessary conditions for an effective category-oriented poisoning attack and devise a black-box strategy to meet both. Let p be a malicious text. For p to reliably compromise all queries $Q \in \mathcal{C}_v$, it must satisfy:

- **Generation condition:** From Eq. 2, the LLM should output $\mathcal{A}^\star$ when p is used as the context for any $Q \in \mathcal{C}_v$. This ensures that even when retrieved alongside other texts, p will bias the RAG toward $\mathcal{A}^\star$.
- **Retrieval condition:** From Eq. 3, p must appear among the top-k retrieved texts for any $Q \in \mathcal{C}_v$. Otherwise, it cannot influence the generated answer. To ensure this, the embedding of p should be close to that of the *category centroid question* Q_c, thereby maximizing semantic similarity to the entire category in the retriever's embedding space.

Since direct joint optimization for both conditions is intractable, we decompose p into two components:

$$p = \text{concat}(p^{(\text{ret})},\ p^{(\text{gen})}), \tag{5}$$

where $p^{(\text{ret})}$ is crafted to maximize retrieval coverage over $\mathcal{C}_v$, and $p^{(\text{gen})}$ is optimized to consistently induce $\mathcal{A}^\star$ during answer generation. The construction of these two sub-texts is described next.

1) Crafting the generation sub-text $p^{(\text{gen})}$: A straightforward method to obtain $p^{(\text{gen})}$ would be gradient-based optimization [18], directly tuning the text towards the target answer $\mathcal{A}^\star$. However, this is infeasible in practice: black-box LLMs (e.g., GPT-4, Llama-2) do not provide gradients, and even with white-box access, the computational cost is prohibitive. To address this, we design a heuristic procedure that avoids gradient computation and leverages the generative power of LLMs.

Specifically, rather than relying on the full set of queries in $\mathcal{C}_v$, we first perform clustering over the query embeddings to partition $\mathcal{C}_v$ into multiple coherent sub-groups. In our experiment, we use K-means method to conduct the clustering step. For each cluster, we identify a representative centroid query that captures the dominant semantics of that sub-class. These centroid queries serve as concise prototypes for the category.

Next, we prompt an auxiliary LLM (not necessarily the same as the victim RAG's LLM) with each centroid query together with the attacker-specified answer $\mathcal{A}^\star$, instructing it to synthesize a malicious text that, when presented as the sole context, consistently drives the LLM to output $\mathcal{A}^\star$. By aggregating the generated texts across multiple clusters, we obtain a generalized $p^{(\text{gen})}$ that preserves coverage across the full semantic range of $\mathcal{C}_v$. This ensures that the crafted poison is not biased toward specific queries but can influence responses broadly within the targeted category.

2) Crafting the retrieval sub-text $p^{(\text{ret})}$: To craft the sub-text $p^{(\text{ret})}$, let f_Q and f_d denote the query and document encoders of a substitute retriever available to the attacker. For queries $Q \in \mathcal{C}_v$, their embeddings $\{f_Q(Q)\}$ form a compact cluster; let c_v be the centroid. We synthesize $p^{(\text{ret})}$ so that $f_d(p^{(\text{ret})})$ lies near c_v, increasing similarity to all queries in the category:

$$p^{(\text{ret})} \approx \arg\max_t \mathbb{E}_{Q\sim\mathcal{C}_v}[\, s(f_Q(Q), f_d(t))\,], \tag{6}$$

where $s(\cdot,\cdot)$ is the similarity function and t is the text. Practically, we approximate c_v by clustering paraphrases of category queries, then compose a semantically central "category prototype" description.

The final poisoned text p is obtained by concatenating $p^{(\text{ret})}$ and $p^{(\text{gen})}$. The retrieval sub-text ensures high retrieval coverage for $\mathcal{C}_v$, while the generation sub-text guarantees that once retrieved, the LLM's output is steered toward $\mathcal{A}^\star$. The complete attack procedure is detailed in Algorithm **??**.

C. Attack Feasibility of CatPoison

We next provide a theoretical justification for the feasibility of *CatPoison* in RAG systems. Let $\mathcal{C}_v$ denote the target semantic category of queries, and let $\mathcal{D}$ be the clean knowledge base. The attack is considered successful if:

$$\forall Q \in \mathcal{C}_v, \quad \mathcal{L}\big(Q, \widehat{\mathcal{R}}(Q,K,\mathcal{D}\cup\mathcal{D}_p)\big) = \mathcal{A}^*,$$

where $\mathcal{D}_p$ is the set of injected poisoned texts and $\mathcal{A}^*$ is the attacker-desired target answer.

1) Retrieval Condition: Consider a retriever equipped with two encoders: a question encoder f_Q and a document encoder

f_d, which are jointly trained. The encoder f_Q maps a query Q into an embedding vector, while f_d produces embeddings for texts in the knowledge base. The relevance between a query Q and a document d is commonly measured by:

$$\text{score}(Q, d) = \cos\big(f_Q(Q), f_d(d)\big).$$

For a target category $\mathcal{C}_v$ whose queries share similar semantics, the set of query embeddings $\{f_Q(Q)\}_{Q\in\mathcal{C}_v}$ will form a compact cluster in the embedding space. By crafting a poisoned document p whose embedding $f_d(p)$ is positioned near the centroid of this region, the retriever is likely to rank p highly for the majority of queries in $\mathcal{C}_v$. This design enables wide retrieval coverage over the entire category.

2) Generation Condition: Poisoned documents in $\mathcal{D}_p$ are further crafted so that, when provided as context to the LLM, they consistently induce the target output:

$$\mathcal{L}(Q, \{p\}) = \mathcal{A}^*, \quad \forall Q \in \mathcal{C}_v,\ p \in \mathcal{D}_p.$$

This can be achieved by embedding explicit or implicit cues in p that bias the LLM's generation toward $\mathcal{A}^*$, regardless of the specific query phrasing within $\mathcal{C}_v$.

Then, the attacker's goal can thus be further formalized as:

$$\max_{\mathcal{D}_p} \frac{1}{|\mathcal{C}_v|} \sum_{Q\in\mathcal{C}_v} \mathbb{I}\left(\mathcal{L}\big(Q, \widehat{\mathcal{R}}(Q, K, \mathcal{D}\cup\mathcal{D}_p)\big) = \mathcal{A}^*\right),$$

where $\mathbb{I}(\cdot)$ is the indicator function. A small $\mathcal{D}_p$ that maximizes this goal implies a successful attack.

3) Feasibility Insight.: Because semantically related queries share a common embedding neighborhood and RAG outputs are highly sensitive to injected context, even a small number of well-crafted poisoned documents can simultaneously satisfy both retrieval and generation conditions. This allows the attack to generalize to unseen queries in $\mathcal{C}_v$ while maintaining stealth, making CatPoison a practical and potent threat in RAG systems.

VI. Experimental evaluation

A. Experimental Setup

1) Datasets: In the following experiments, we evaluate our approach on three benchmark question-answering datasets: *Natural Questions (NQ)* [19], *HotpotQA* [20], and *MS-MARCO* [21]. Each accompanied by its own knowledge database. For NQ and HotpotQA, the databases are derived from Wikipedia and contain 2,681,468 and 5,233,329 texts, respectively. In contrast, the MS-MARCO database, comprising 8,841,823 texts, is built from web documents retrieved via the Microsoft Bing search engine [22]. Each dataset is further paired with a corresponding set of questions.

2) RAG Setup: Recall that a RAG system consists of three key components: the *knowledge database*, *retriever*, and *LLM*, configured as follows:

- *Knowledge database:* For each dataset, we directly use its corresponding knowledge base in the RAG pipeline, resulting in three distinct databases.
- *Retriever:* We evaluate the Contriever retriever [23], Contriever-ms (fine-tuned on MS-MARCO) [23], and ANCE [24], using the dot product between question and text embeddings as the default similarity metric [9]. The impact of this similarity measure is further analyzed in our evaluation.
- *LLM:* Five LLMs are considered for performance evaluation: GPT-3.5 [25], GPT-4.1 [26], Llama-7B [27], Mistral-7B [28], and Vicuna-7b [29].

3) Evaluation Metrics: For evaluating the performance of *CatPoison*, we adopt the following standard metrics:

- *Attack Success Rate (ASR).* We employ the ASR to quantify the proportion of questions from the target class answered with the attacker-desired responses. Following prior work [30], we consider two answers equivalent for close-ended questions if the target answer appears as a substring in the LLM's generated output under attack, a criterion we refer to as *substring matching*. Exact Match is not adopted, as it can be overly strict. For example, it would treat "Donald Trump" and "The president of US is Donald Trump" as different responses to "Who is the president of US?".
- *F1-Score.* We use the *F1-Score* to assess whether injected malicious texts are retrieved for target class questions. In RAG, the system retrieves the top-k texts for each question. F1-Score, computed as $2 \cdot \text{Precision} \cdot \text{Recall}/(\text{Precision}+\text{Recall})$, captures the balance between Precision and Recall. A higher value of F1-Score indicate that more malicious texts are successfully retrieved.
- *Accuracy (Acc).* Accuracy is defined as the fraction of testing queries from a victim class are misclassified to the attacker-desired class.

4) Baselines: To the best of our knowledge, no existing attack directly achieve class-oriented poisoning attack. For comparison, we adapt prior attacks [7], [9] to our attack scenario, yielding the following baselines:

- *PoisonedRAG* [7]: This is the state-of-the-art targeted knowledge poisoning attack in RAG. We extend it to a class-oriented setting by injecting poisoned texts for all questions in a single step within the target category.
- *Corpus Poisoning Attack (CPA)* [9]: This method inserts malicious texts into the knowledge base so that they can be retrieved for arbitrary queries, requiring white-box access to the retriever. We use the default implementation [9] in our experiments.

B. Attack Performance

In this section, we evaluate the performance of the proposed attack on RAG systems across different datasets and LLMs. As shown in Table I, our method achieves an average ASR of 0.801 across all settings, with a mean retrieval-side F1 score of 0.939 for poisoned texts, substantially outperforming both baseline attacks. In extreme cases, 6 out of 15 settings yield ASR $\geq$ 0.90, including 3 settings that achieve 100% ASR,

TABLE I: CatPoison achieves high ASRs (%) across three datasets and five different LLMs by injecting identical malicious texts for a category-oriented question into knowledge corpus. Precision and Recall are omitted as they are identical to the F1-Score.

Dataset	Attack	Metrics	LLMs of RAG				
			GPT-3.5	GPT-4.1	Llama-7B	Vicuna-7B	Mistral-7B
NQ	CPA	ASR	0.01	0.01	0.01	0.01	0.01
		F1-Score	0.932				
	PoisonedRAG	ASR	0.216	0.216	0.216	0.216	0.216
		F1-Score	0.932				
	CatPoison	ASR	0.7	0.875	1.0	0.9	0.925
		F1-Score	0.932				
HotpotQA	CPA	ASR	0.01	0.01	0.01	0.01	0.01
		F1-Score	1.0				
	PoisonedRAG	ASR	0.217	0.217	0.217	0.217	0.217
		F1-Score	1.0				
	CatPoison	ASR	0.756	0.795	1.0	0.545	0.951
		F1-Score	1.0				
MS-MARCO	CPA	ASR	0.03	0.03	0.03	0.03	0.03
		F1-Score	0.914				
	PoisonedRAG	ASR	0.26	0.26	0.26	0.26	0.26
		F1-Score	0.914				
	CatPoison	ASR	0.50	0.571	1	0.785	0.714
		F1-Score	0.914				

demonstrating that the attack can reliably force RAG outputs to the target answer under favorable conditions. Furthermore, ASR shows a moderately positive correlation with the retrieval F1 score, indicating that stable retrieval of poisoned documents is a critical determinant of attack success.

The susceptibility of different LLMs shows clear but non-monotonic variation, which can be attributed to differences in model capability structure, reliance on retrieved evidence, and safety alignment strategies. When aggregated by model, the mean ASR is as follows: LLaMA-7B at 1.000, Mistral-7B at 0.863, GPT-4.1 at 0.747, Vicuna-7B at 0.743, and GPT-3.5 at 0.652. Meanwhile, F1 scores are consistently high and closely distributed across models (mostly within the range of 0.905–0.949), indicating that differences in the steerability of the generation component largely account for the ASR gap. Specifically, different LLM models tend to rely more heavily on contextual evidence retrieved during inference and perform less long-range self-checking or conflict resolution. Once the retrieval stage consistently supplies "category-consistent" poisoned texts, these models can reproduce the same incorrect prior across diverse paraphrases within the category. In contrast, more capable and better-aligned models (e.g., GPT-4.1) demonstrate a degree of resilience despite achieving similarly high F1 scores; while their ASR remains significantly higher than that of traditional baselines, it is lower than that of more evidence-compliant open-weight models. This phenomenon indirectly validates our attack mechanism: when retrieval-side category coverage is sufficiently stable, the susceptibility of the generation component becomes the limiting factor.

At the dataset level, we observe an interaction between retrieval accessibility and question style. Aggregated across the corpus, NQ achieves the highest ASR while maintaining a fixed F1 of 0.932; HotpotQA achieves the highest F1 but only a moderate ASR; MS MARCO proves the most challenging. This suggests that a high F1 score is a necessary but not sufficient condition.

In summary, category-oriented poisoning offers three major advantages over traditional instance-specific poisoning: (i) by encoding category anchors into poisoned texts and ensuring cross-paraphrase retrieval consistency, it significantly expands the retrieval coverage of the attack; (ii) given stable coverage, the generation model's reliance on retrieved evidence allows category priors to repeatedly influence answer generation, thereby pushing the ASR above 0.80 in a steady state and achieving 1.00 in several attack settings; and (iii) the absolute improvement over baseline methods demonstrates that shifting the poisoning target from *single data* to *surfaces* is the key mechanism driving the observed performance leap.

C. Impact of Retriever

To evaluate the generalization ability of the *CatPoison* attack under different retriever models, we conduct experiments by fixing the number of retrieved documents and poisoned texts while replacing the retriever module (i.e., Contriever, Contriever-MS, and ANCE).

The results in Table II show that the ASR largely depends on the type of retriever used in the target RAG system. In most tested scenarios, when Contriever is used as the retriever, the ASR reaches the highest level. However, when the retriever is replaced with Contriever-MS or ANCE, the attack effectiveness generally decreases. For instance, under the combination of Llama-7B and the NQ dataset, the ASR drops from 1.0 (Contriever) to 0.9 (Contriever-MS) and 0.825 (ANCE). This trend is consistent across all model-dataset com-

TABLE II: Impact of Retriever.

Retriever	Dataset	LLMs of RAG				
		GPT-3.5	GPT-4.1	Llama-7B	Vicuna-7B	Mistral-7B
Contriever	NQ	0.7	0.875	1.0	0.9	0.925
	HotpotQA	0.756	0.795	1.0	0.545	0.951
	MS-MARCO	0.571	0.5	1.0	0.785	0.714
Contriever-ms	NQ	0.45	0.425	0.9	0.825	0.80
	HotpotQA	0.659	0.773	0.927	0.744	0.659
	MS-MARCO	0.5	0.571	0.929	0.714	0.643
ANCE	NQ	0.175	0.325	0.825	0.65	0.575
	HotpotQA	0.463	0.682	0.756	0.22	0.561
	MS-MARCO	0.357	0.357	0.857	0.785	0.571

TABLE III: Overall Performance.

Dataset	**LLM**	**Metric**			
		$Acc_o(\%)$	$Acc(\%)$	Precision	Recall
NQ	GPT-3.5	0.849	0.826	0.932	0.932
	GPT-4.1	0.766	0.739	0.932	0.932
	Llama-7B	0.977	0.955	0.932	0.932
	Vicuna-7B	0.773	0.727	0.932	0.932
	Mistral-7B	1.000	0.977	0.932	0.932
HotpotQA	GPT-3.5	0.813	0.794	1.0	1.0
	GPT-4.1	0.759	0.722	1.0	1.0
	Llama-7B	0.962	0.885	1.0	1.0
	Vicuna-7B	0.759	0.741	1.0	1.0
	Mistral-7B	0.923	0.885	1.0	1.0
MS-MARCO	GPT-3.5	0.867	0.867	0.914	0.914
	GPT-4.1	0.838	0.838	0.914	0.914
	Llama-7B	0.836	0.836	0.914	0.914
	Vicuna-7B	0.833	0.833	0.914	0.914
	Mistral-7B	0.85	0.85	0.914	0.914

binations, confirming that the attack performance is sensitive to the choice of retriever.

It is worth noting that the ASR on ANCE is consistently the lowest among the three retrievers. This can be attributed to the substantial architectural and training differences between ANCE and Contriever, which hinder the transferability of attacks from Contriever to ANCE. Moreover, the lack of fine-tuned parameter configurations for ANCE may further degrade its retrieval performance, reducing the overall ASR.

D. Overall Performance

In this section, we aim to assess the extent to which our poisoning attack affects model performance on non-target question domains. We injected poisoned samples into a set of mainstream LLMs, including GPT-4.1, GPT-3.5, LLaMA-7B, Vicuna-7B, and Mistral-7B, and systematically evaluated their accuracy on three standard QA datasets before (i.e., Acc_o) and after (i.e., Acc) the attack. This allows us to measure both the precision of the attack and its potential side effects in terms of generalization.

The results in Table III reveal a consistent and noteworthy phenomenon. The most striking observation comes from the MS MARCO dataset, where all tested LLMs exhibited no observable change in QA accuracy after the poisoning attack, with pre- and post-attack performance remaining identical. This strongly indicates that the proposed poisoning attack possesses a high degree of target specificity. The attack precisely impacts the model's ability to handle a particular category of questions, without causing measurable collateral damage to its performance in other domains or knowledge areas represented by MS MARCO. This finding, observed across a wide range of models, validates the precision of the attack vector and suggests a certain degree of modularity or compartmentalization in the internal knowledge and capabilities of LLMs.

In contrast, small fluctuations were observed on HotpotQA and NQ. Across all models, accuracy on these datasets decreased slightly, with an overall average change of -3.30% after the attack. It is important to emphasize that such a degree of change is minimal and falls within an acceptable range. Considering the inherent stochasticity of LLMs, where identical inputs may yield different outputs, part of this fluctuation is likely attributable to the model's natural randomness rather than solely to the negative generalization effects of the poisoning. Among all models, GPT-3.5 exhibited the strongest robustness with the smallest change (-2.10%), followed by Mistral-7B (-3.05%), GPT-4.1 (-3.20%), and Vicuna-7B (-3.20%), while LLaMA-7B showed the largest fluctuation (-4.95%).

In summary, the experiment in Table III clearly demonstrates the dual nature of data poisoning attacks on LLMs. On one hand, such attacks can achieve high precision, fulfilling their specific objectives without impairing model performance on certain general-purpose tasks (e.g., MS MARCO). On the other hand, the side effects are minimal, manifesting only as small and acceptable performance drops on certain related tasks, which can be partially explained by the inherent randomness of LLM generation. From an overall performance perspective, mainstream LLMs exhibit strong robustness against such targeted attacks, with their general capabilities remaining largely unaffected. These findings provide valuable insights for understanding the security boundaries of LLMs and for designing more resilient AI systems.

E. Impact of the Number of Poisoned texts

In this section, we evaluate the impact of the number of injected poisoned texts on the ASR of *CatPoison*. The

TABLE IV: Impact of the Number of Poisoned texts.

Dataset	Number			
	10	20	10	20
NQ	0.275	0.575	0.65	0.80
HotpotQA	0.5	0.727	0.75	0.773
MS-MARCO	0.357	0.643	0.714	0.714

experimental setup fixes GPT-4.1 as the target LLM and keeps the number of retrieved documents constant at 20. Under this setting, we vary the number of injected poisoned texts and observe the changes in the ASR across three benchmark datasets, NQ, HotpotQA, and MS-MARCO, to quantitatively explore the relationship between attack intensity and the scale of poisoned samples.

The results in Table IV clearly indicate a strong positive correlation between the number of poisoned texts and the attack success rate. As the number of injected malicious texts increases from 10 to 40, a consistent rise in ASR is observed across all three datasets. For instance, on the MS-MARCO dataset, ASR rises from 0.275 to 0.800, representing nearly a threefold improvement. Similarly, on the NQ and HotpotQA datasets, ASR steadily increases from 0.500 and 0.357 to 0.773 and 0.714, respectively. This trend strongly demonstrates that increasing the absolute number of poisoned samples is a key factor in enhancing the effectiveness of category-oriented knowledge poisoning attacks.

F. Impact of k

In this section, we investigate the impact of the number of retrieved documents (i.e., the k value) on the success rate of data poisoning attacks in RAG systems. Experimental results reveal a nonlinear and dataset-dependent relationship between the attack success rate and k. We conducted experiments using GPT-4.1 on three public QA datasets with k values ranging from 5 to 25.

The findings in Fig. 2 indicate that the attack success rate does not monotonically increase with larger k. On MS MARCO, the success rate exhibited a stepwise upward trend, rising sharply from 0.571 at $k = 10$ to 0.714 at $k = 15$, after which it plateaued. This suggests that, in this setting, sufficiently large k values can increase the proportion of poisoned texts within the context, thereby boosting attack success until a saturation point is reached. In contrast, results on NQ demonstrated a different pattern: the success rate peaked at 0.875 when $k = 10$, but declined as k increased further. This supports our hypothesis that retrieving too many documents may introduce benign evidence contradictory to or irrelevant to the poisoned content. Such benign documents dilute the influence of poisoned texts, and as the model processes richer yet more conflicting information, its internal conflict resolution mechanisms may be triggered, making it more likely to follow benign guidance and thus causing the attack to fail.

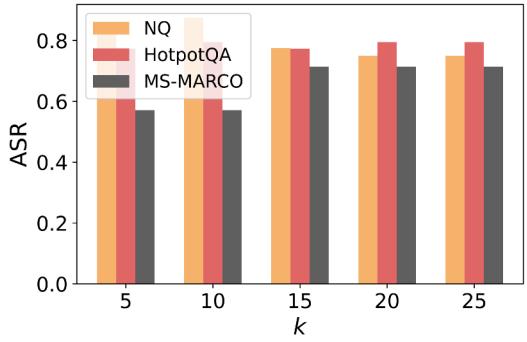

Fig. 2: Impact of k.

Moreover, very small k values also limit attack effectiveness. With too few retrieval slots, the system may return highly relevant but unpoisoned documents. Even though the poisoned texts are designed to align closely with the target query, the lack of dominance in the retrieved set means that benign documents can still guide the model toward harmless responses, leading to failed attacks. Results on HotpotQA were relatively stable, with success rates fluctuating only between 0.77 and 0.80. This suggests that, under the characteristics of this dataset, the effectiveness of poisoning is less sensitive to variations in k.

In summary, the number of retrieved documents (i.e., k) is a critical hyperparameter shaping the vulnerability of RAG systems to data poisoning attacks. Attackers cannot always maximize success by simply enlarging k; instead, the success rate appears to depend on an "optimal attack window." Specifically, k must be large enough to ensure sufficient exposure of poisoned texts, but small enough to avoid introducing excessive contradictory benign evidence. This finding points to a promising defensive direction for RAG systems: dynamically adjusting or optimizing k could potentially reduce the security risks posed by data poisoning in certain scenarios.

VII. Possible Countermeasures

Since our attack goal is to investigate the adversarial vulnerabilities introduced by category-oriented poisoning attacks in RAG systems, and there are currently very few defenses explicitly designed for such threats. Thus, it is important to outline potential countermeasures that could mitigate their impact. From the data perspective, incoming and updated texts in the knowledge corpus should be subjected to rigorous provenance verification and semantic consistency checks. Automated fact-checking, cross-source validation, and embedding-based anomaly detection can help identify poisoning texts targeting an entire semantic category [6], [17].

From the retrieval and generation perspective, increasing retrieval diversity through stochastic candidate selection and multi-model re-ranking can substantially reduce the influence of a single malicious text on the final answer. Complementary

to this, integrating evidence-based generation validation, where the model compares responses derived from different retrieval subsets can help detect and neutralize biases injected through poisoned texts [4], [7].

VIII. Conclusion

In this paper, we introduce *CatPoison*, the first category-oriented poisoning attack against RAG systems. Unlike prior instance-level approaches that focus on manipulating outputs for specific queries, *CatPoison* poses a broader and more practical threat by corrupting the RAG's behavior across an entire semantic category through the injection of only a few strategically crafted poisoned documents. Building on insights from existing knowledge corruption attacks in RAG, we formulate *CatPoison* as an optimization problem for generating generalized poisoned texts under a black-box threat model. Experimental results show that *CatPoison* consistently outperforms baselines in attack success rate, while preserving stealth and incurring minimal resource overhead.

References

[1] W. Su, Q. Ai, J. Zhan, Q. Dong, and Y. Liu, "Dynamic and parametric retrieval-augmented generation," in *Proceedings of SIGIR*, 2025, pp. 4118–4121.

[2] W. Su, Y. Tang, Q. Ai, J. Yan, C. Wang, H. Wang, Z. Ye, Y. Zhou, and Y. Liu, "Parametric retrieval augmented generation," in *Proceedings of SIGIR*, 2025, pp. 1240–1250.

[3] W. Fan, Y. Ding, L. Ning, S. Wang, H. Li, D. Yin, T.-S. Chua, and Q. Li, "A survey on rag meeting llms: Towards retrieval-augmented large language models," in *Proceedings of KDD*, 2024, pp. 6491–6501.

[4] C. Wang, Y. Wang, Y. Cai, and B. Hooi, "Tricking retrievers with influential tokens: An efficient black-box corpus poisoning attack," in *Proceedings of ACL*, 2025, pp. 4183–4194.

[5] Y. Jiao, X. Wang, and K. Yang, "Pr-attack: Coordinated prompt-rag attacks on retrieval-augmented generation in large language models via bilevel optimization," in *Proceedings of SIGIR*, 2025.

[6] A. Shafran, R. Schuster, and V. Shmatikov, "Machine against the rag: Jamming retrieval-augmented generation with blocker documents," in *Proceedings of USENIX Security*, 2025.

[7] W. Zou, R. Geng, B. Wang, and J. Jia, "Poisonedrag: Knowledge corruption attacks to retrieval-augmented generation of large language models," in *Proceedings of USENIX Security*, 2025.

[8] Z. Chen, Z. Xiang, C. Xiao, D. Song, and B. Li, "Agentpoison: Red-teaming llm agents via poisoning memory or knowledge bases," in *Proceedings of NeurIPS*, vol. 37, 2024, pp. 130 185–130 213.

[9] Z. Zhong, Z. Huang, A. Wettig, and D. Chen, "Poisoning retrieval corpora by injecting adversarial passages," in *Proceedings of EMNLP*, 2023, pp. 13 764–13 775.

[10] Y. Hou, A. Pascale, J. Carnerero-Cano, T. Tchrakian, R. Marinescu, E. Daly, I. Padhi, and P. Sattigeri, "Wikicontradict: A benchmark for evaluating llms on real-world knowledge conflicts from wikipedia," in *Proceedings of NeurIPS*, vol. 37, 2024, pp. 109 701–109 747.

[11] C. Wang, J. Chen, Y. Yang, X. Ma, and J. Liu, "Poisoning attacks and countermeasures in intelligent networks: Status quo and prospects," *Digital Communications and Networks*, vol. 8, no. 2, pp. 225–234, 2022.

[12] J. Chen, Y. Gao, J. Shan, K. Peng, C. Wang, and H. Jiang, "Manipulating supply chain demand forecasting with targeted poisoning attacks," *IEEE transactions on industrial informatics*, vol. 19, no. 2, pp. 1803–1813, 2022.

[13] B. Biggio, B. Nelson, and P. Laskov, "Poisoning attacks against support vector machines," in *Proceedings of ICML*, 2012.

[14] M. Jagielski, A. Oprea, B. Biggio, C. Liu, C. Nita-Rotaru, and B. Li, "Manipulating machine learning: Poisoning attacks and countermeasures for regression learning," in *Proceedings of IEEE Symposium on Security and Privacy (S&P)*, 2018.

[15] B. Zhao and Y. Lao, "Towards class-oriented poisoning attacks against neural networks," in *Proceedings of the IEEE/CVF Winter Conference on Applications of Computer Vision*, 2022, pp. 3741–3750.

[16] J. Chen, J. Wu, H. Yin, Q. Li, W. Zhang, and C. Wang, "Class-targeted poisoning attacks against dnns," in *2023 IEEE 22nd International Conference on Trust, Security and Privacy in Computing and Communications (TrustCom)*, 2023, pp. 20–27.

[17] N. Carlini, M. Jagielski, C. A. Choquette-Choo, D. Paleka, W. Pearce, H. Anderson, A. Terzis, K. Thomas, and F. Tramèr, "Poisoning web-scale training datasets is practical," in *2024 IEEE Symposium on Security and Privacy (SP)*. IEEE, 2024, pp. 407–425.

[18] J. Ebrahimi, A. Rao, D. Lowd, and D. Dou, "Hotflip: White-box adversarial examples for text classification," in *Proceedings of ACL*, 2018, pp. 31–36.

[19] T. Kwiatkowski, J. Palomaki, O. Redfield, M. Collins, A. Parikh, C. Alberti, D. Epstein, I. Polosukhin, J. Devlin, K. Lee *et al.*, "Natural questions: a benchmark for question answering research," *TACL*, vol. 7, pp. 452–466, 2019.

[20] Z. Yang, P. Qi, S. Zhang, Y. Bengio, W. Cohen, R. Salakhutdinov, and C. D. Manning, "Hotpotqa: A dataset for diverse, explainable multi-hop question answering," in *Proceedings of EMNLP*, 2018.

[21] T. Nguyen, M. Rosenberg, X. Song, J. Gao, S. Tiwary, R. Majumder, and L. Deng, "Ms marco: A human generated machine reading comprehension dataset," *choice*, vol. 2640, p. 660, 2016.

[22] "Ms-marco microsoft bing," https://microsoft.github.io/msmarco/.

[23] G. Izacard, M. Caron, L. Hosseini, S. Riedel, P. Bojanowski, A. Joulin, and E. Grave, "Unsupervised dense information retrieval with contrastive learning," *Transactions on Machine Learning Research*, 2022.

[24] L. Xiong, C. Xiong, Y. Li, K.-F. Tang, J. Liu, P. N. Bennett, J. Ahmed, and A. Overwijk, "Approximate nearest neighbor negative contrastive learning for dense text retrieval," in *Proceedings of ICLR*, 2021.

[25] L. Ouyang, J. Wu, X. Jiang, D. Almeida, C. Wainwright, P. Mishkin, C. Zhang, S. Agarwal, K. Slama, A. Ray *et al.*, "Training language models to follow instructions with human feedback," in *Proceedings of NeurIPS*, vol. 35, 2022, pp. 27 730–27 744.

[26] OpenAI, "Introducing gpt-4.1 in the api," https://openai.com/index/gpt-4-1/, 2025.

[27] H. Touvron, L. Martin, K. Lu *et al.*, "Llama 3: Open foundation and instruction models," 2024, meta AI Technical Report. [Online]. Available: https://ai.meta.com/llama/

[28] A. M. C. B. D. S. C. D. d. l. C. F. B. G. L. G. L. L. S. L. R. L. M.-A. L. P. S. T. L. S. T. L. T. W. T. L. Albert Q. Jiang, Alexandre Sablayrolles and W. E. Sayed, "Mistral 7b," *CoRR arxiv: 2310.06825*, 2023.

[29] W.-L. Chiang, Z. Li, Z. Lin, Y. Sheng, Z. Wu, H. Zhang, L. Zheng, S. Zhuang, Y. Zhuang, J. E. Gonzalez *et al.*, "Vicuna: An open-source chatbot impressing gpt-4 with 90%* chatgpt quality," *See https://vicuna. lmsys. org (accessed 14 April 2023)*, vol. 2, no. 3, p. 6, 2023.

[30] Y. Huang, S. Gupta, M. Xia, K. Li, and D. Chen, "Catastrophic jailbreak of open-source llms via exploiting generation," *arXiv*, 2023.